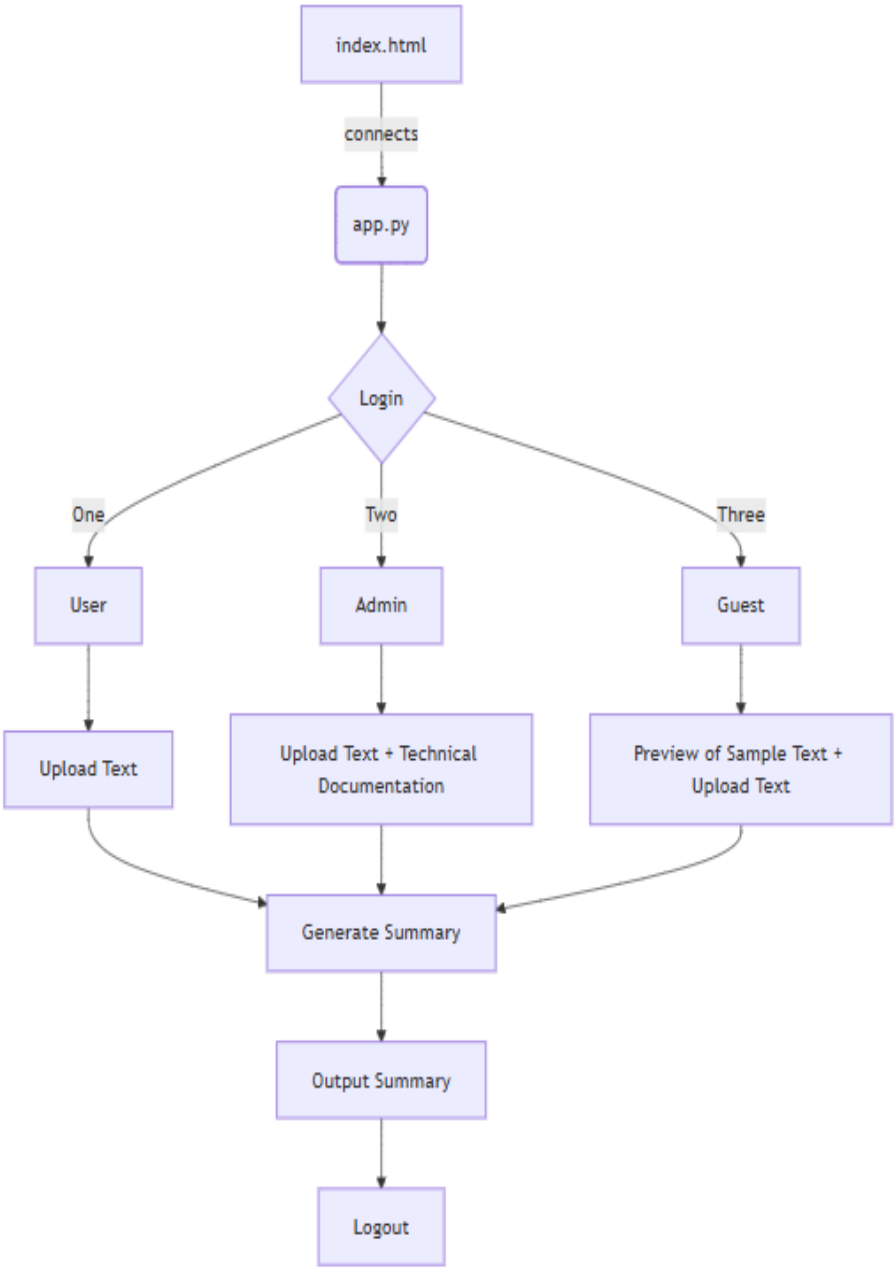
FLOWCHART:


index.html
connects
app.py
Login
One
Two
Three
User
Admin
Guest
Upload Text
Upload Text + Technical Documentation
Preview of Sample Text + Upload Text
Generate Summary
Output Summary
Logout

SOURCE CODE:

index.html(Frontend)

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>AI Text Summarizer</title>
    <link rel="icon" type="image/x-icon" href="../Backend/static/favicon.ico">

    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }

        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            color: #333;
            background-image: url("bgi3.jpeg");
            background-size: cover;
            background-position: center;
            background-repeat: no-repeat;
            background-attachment: fixed;
            height: 100vh;
        }

        .container {
            max-width: 1200px;
            margin: 0 auto;
            padding: 20px;
        }

        .login-modal {
            display: none;
            position: fixed;
            top: 0;
            left: 0;
            width: 100%;
            height: 100%;
            background: rgba(0, 0, 0, 0.8);
            z-index: 1000;
            backdrop-filter: blur(5px);
        }

        .login-box {
            position: absolute;
            top: 50%;
            left: 50%;
            transform: translate(-50%, -50%);
            background: white;
            padding: 40px;
            border-radius: 20px;
            box-shadow: 0 20px 40px rgba(0, 0, 0, 0.3);
            width: 90%;
            max-width: 400px;
            text-align: center;
```

```
}

.login-btn {
    width: 100%;
    padding: 15px;
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    color: white;
    border: none;
    border-radius: 12px;
    font-size: 16px;
    font-weight: 600;
    cursor: pointer;
}

.login-btn:hover {
    transform: translateY(-3px);
    box-shadow: 0 12px 25px rgba(102, 126, 234, 0.4);
}

.app {
    display: none;
    background: white;
    border-radius: 20px;
    box-shadow: 0 20px 40px rgba(0, 0, 0, 0.1);
    overflow: hidden;
    margin-top: 20px;
    position: relative;
    padding-bottom: 70px;
    /* □ prevents overlap */
}

.header {
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    color: white;
    padding: 30px;
    text-align: center;
}

.content {
    padding: 40px;
}

.summary-options {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
    gap: 50px;
    margin-bottom: 30px;
}

.option-card {
    padding: 20px;
    border: 2px solid #e2e8f0;
    border-radius: 12px;
    text-align: center;
    cursor: pointer;
}

.option-card.active {
    border-color: #667eea;
    background: rgba(102, 126, 234, 0.1);
}
```

```
        .option-card:hover {
            transform: translateY(-4px);
            border-color: #667eea;
        }

        textarea,
        input[type="file"] {
            width: 100%;
            padding: 15px;
            border: 2px solid #e2e8f0;
            border-radius: 12px;
            font-size: 14px;
        }

        textarea {
            min-height: 150px;
        }

        .generate-btn {
            width: 100%;
            padding: 18px;
            background: linear-gradient(135deg, #48bb78 0%, #38a169 100%);
            color: white;
            border: none;
            border-radius: 12px;
            font-size: 18px;
            font-weight: 600;
            cursor: pointer;
        }

        .generate-btn:hover:not(:disabled) {
            transform: translateY(-3px);
            box-shadow: 0 10px 25px rgba(72, 187, 120, 0.4);
        }

        .results {
            display: none;
            margin-top: 30px;
        }

        .results-card {
            background: #f8fafc;
            border: 2px solid #e2e8f0;
            border-radius: 12px;
            padding: 25px;
        }

        .results-text {
            line-height: 1.6;
            white-space: pre-wrap;
        }

        .loading {
            display: inline-block;
            width: 20px;
            height: 20px;
            border: 3px solid #f3f3f3;
            border-top: 3px solid #667eea;
            border-radius: 50%;
            animation: spin 1s linear infinite;
            margin-right: 10px;
        }
```

```
        @keyframes spin {
            0% {
                transform: rotate(0);
            }

            100% {
                transform: rotate(360deg);
            }
        }
    </style>
</head>

<body>

    <!-- LOGIN -->
    <div class="login-modal" id="loginModal">

        <div class="login-box">
            <h1>Text Summarizer</h1>
            <input type="text" id="username" placeholder="Username"
                style="margin-bottom:20px;padding:15px;width:100%;border:2px solid #e2e8f0;border-radius:12px;">
            <input type="password" id="password" placeholder="Password"
                style="margin-bottom:20px;padding:15px;width:100%;border:2px solid #e2e8f0;border-radius:12px;">
            <button class="login-btn" onclick="login()">Login</button>
        </div>
    </div>

    <div class="container">
        <div class="app" id="app">

            <div class="header">
                <img src="../Backend/static/logo1.png" alt="App Logo" style="width: 90px;margin-bottom: 15px;">
                <h1>Text Summarizer using Artifical Intelligence</h1>
                <h2 id="greetingMessage" style="margin-bottom:10px;">
                    </style>
                </h2>
                <!-- USER MODE DISPLAY -->
                <h3 id="modeDisplay" style="font-weight:500;color:#f0f0f0;">
                </h3>
                <!-- Demo + Logout -->
                <div style="position:absolute;top:20px;left:20px;">
                    <button onclick="showDemo()"
                        style="padding:8px 15px;border:none;border-
radius:8px;background:#fff;color:#333;cursor:pointer;">
                        <img src="demovid.png"> Demo Video
                    </button>
                </div>

                <div style="position:absolute;top:20px;right:20px;">
                    <button onclick="logout()"
                        style="padding:8px 15px;border:none;border-
radius:8px;background:#ff4d4d;color:white;cursor:pointer;">
                        <img src="logout (3).png"> Logout
                    </button>
                </div>
            </div>

            <div class="content">
                <input type="file" id="fileInput" accept=".txt">
                <br><br>
```

```
        <textarea id="textInput" placeholder="Paste your text here"></textarea>
        <br><br>

        <div class="summary-options">
            <div class="option-card active" data-type="short">
                <h3><img src="paper.png"> Short</h3>
            </div>
            <div class="option-card" data-type="medium">
                <h3><img src="book1.png"> Medium</h3>
            </div>
            <div class="option-card" data-type="large">
                <h3><img src="stackofbooks.png"> Large</h3>
            </div>
        </div>

        <button class="generate-btn" onclick="generateSummary()" id="generateBtn">
            Generate Summary
        </button>

        <div class="results" id="results"></div>
        <!-- GUEST PREVIEW SECTION -->
        <div id="previewSection"
            style="display:none;margin-top:30px;background:#f1f5f9;padding:20px;border-radius:10px;">
            <h3>Mode : Guest</h3>
            <p>You are viewing a Sample Text. You can upload your own file to generate Summary.</p>
        </div>
        <!-- ADMIN TECHNICAL DOCUMENT SECTION -->
        <div id="technicalDocSection"
            style="display:none;margin-top:40px;border-top:2px solid #e2e8f0;padding-top:20px;">
            <h3>Technical Document (Admin Only)</h3>
            <input type="file" accept=".pdf" id="technicalPdfUpload">
            <p style="font-size:13px;color:#666;margin-top:5px;">
                Upload technical PDF documents (Admin Access Only)
            </p>
        </div>

    </div>
    <!-- Bottom Bar (About + Version) -->
    <div style="position:absolute;
    bottom:15px;
    left:20px;
    right:20px;
    display:flex;
    justify-content:space-between;
    align-items:center;">

        <!-- About + Technical Document Buttons -->
        <div style="display:flex;gap:10px;align-items:center;">

            <button onclick="openAbout()" style="padding:8px 15px;
            border:none;
            border-radius:8px;
            background:#667eea;
            color:white;
            cursor:pointer;">
                About
            </button>

            <button id="techDocBtn" onclick="openTechDoc()" style="padding:8px 15px;
            border:none;
            border-radius:8px;
            background:#667eea;
```

```
                    color:white;
                    cursor:pointer;
                    display:none;">
                        Technical Document
                    </button>

                </div>

                <!-- Version -->
                <span style="font-size:13px;color:#666;font-weight:500;">
                    Ver 1.0
                </span>

            </div>
        </div>
    </div>

    <!-- POPUP -->
    <div id="popupModal"
        style="display:none;position:fixed;top:0;left:0;width:100%;height:100%;background:rgba(0,0,0,0.6);z-
index:2000;">
        <div
            style="position:absolute;top:50%;left:50%;transform:translate(-50%,-
50%);background:white;padding:30px;border-radius:15px;text-align:center;">
            <h3>Generating Summary...</h3>
            <p>Please wait while AI processes your text.</p>
        </div>
    </div>

    <!-- DEMO VIDEO -->
    <div id="demoModal"
        style="display:none;position:fixed;top:0;left:0;width:100%;height:100%;background:rgba(0,0,0,0.7);z-
index:2000;">
        <div
            style="position:absolute;top:50%;left:50%;transform:translate(-50%,-
50%);background:white;padding:20px;border-radius:15px;width:80%;max-width:600px;text-align:center;">
            <h3>Demo Video</h3>
            <video width="100%" controls>
                <source src="demo recording.mp4" type="video/mp4">
            </video>
            <br><br>
            <button onclick="closeDemo()"
                style="padding:8px 20px;background:#667eea;color:white;border:none;border-
radius:8px;">Close</button>
        </div>
    </div>
    <!-- ABOUT MODAL -->
    <div id="aboutModal"
        style="display:none;position:fixed;top:0;left:0;width:100%;height:100%;background:rgba(0,0,0,0.7);z-
index:2000;">
        <div style="position:absolute;top:50%;left:50%;transform:translate(-50%,-50%);
        background:white;padding:20px;border-radius:15px;width:80%;max-width:700px;text-align:center;">

            <h3>About This Project</h3>

            <!-- Project Description Image -->
            <img src="about_jpg.jpg" alt="Project Description" style="width:100%;max-height:800px;object-
fit:contain;
                    border-radius:10px;border:1px solid #ddd;margin-top:15px;">

            <br><br>
            <button onclick="closeAbout()"
```

```
                style="padding:8px 20px;background:#667eea;color:white;border:none;border-radius:8px;">
                Close
            </button>
        </div>
    </div>
    <!-- TECHNICAL DOCUMENT MODAL -->
    <div id="techDocModal"
        style="display:none;position:fixed;top:0;left:0;width:100%;height:100%;background:rgba(0,0,0,0.7);z-
index:2000;">

        <div
            style="position:absolute;top:50%;left:50%;transform:translate(-50%,-
50%);background:white;padding:20px;border-radius:15px;width:80%;max-width:800px;text-align:center;">

            <h3>Technical Document</h3>

            <iframe src="Technical Documentation.docx" width="100%"
                height="450" style="border:none;">
            </iframe>

            <br><br>

            <button onclick="closeTechDoc()"
                style="padding:8px 20px;background:#667eea;color:white;border:none;border-radius:8px;">
                Close
            </button>

        </div>
    </div>
    <br><br>
    </div>
    </div>

    <script>

        const users = {
            user: { password: 'password', role: 'User' },
            admin: { password: 'admin123', role: 'Admin' },
            guest: { password: 'guest123', role: 'Guest' }
        };

        let currentRole = null;
        // Greeting
        window.onload = function () {
            const hour = new Date().getHours();
            let greet = "Welcome!";
            if (hour < 12) greet = "Good Morning...!";
            else if (hour < 18) greet = "Good Afternoon! ";
            else greet = "Good Evening.! ";
            document.getElementById("greetingMessage").innerText = greet;
            document.getElementById('loginModal').style.display = 'block';
        };

        function login() {
            const u = document.getElementById('username').value;
            const p = document.getElementById('password').value;

            if (users[u] && users[u].password === p) {
                currentRole = users[u].role;

                document.getElementById('loginModal').style.display = 'none';
                document.getElementById('app').style.display = 'block';
```

```
                document.getElementById("modeDisplay").innerText = "Mode : " + currentRole;

                applyRolePermissions();
            } else {
                alert('Invalid credentials! Use: user / password OR admin / password OR guest / password');
            }
        }

        function logout() {
            document.getElementById('app').style.display = 'none';
            document.getElementById('loginModal').style.display = 'block';
            document.getElementById('results').style.display = 'none';
            document.getElementById('textInput').value = '';
            document.getElementById("modeDisplay").innerText = "";
            document.getElementById("techDocBtn").style.display = "none";
        }

        function applyRolePermissions() {

            // Reset everything first
            document.getElementById('textInput').disabled = false;
            document.getElementById('fileInput').disabled = false;
            document.getElementById('generateBtn').style.display = 'block';
            document.getElementById('technicalDocSection').style.display = 'none';
            document.getElementById('previewSection').style.display = 'none';

            if (currentRole === "Admin") {
                // Hide old section
                document.getElementById('technicalDocSection').style.display = 'none';

                // Show Technical Document button
                document.getElementById("techDocBtn").style.display = "inline-block";
            }

            if (currentRole === "Guest") {
                // Guest gets preview-only default content
                document.getElementById('textInput').value = "The operating system, or OS, as we will often call
it, is the intermediary between users and the computer system.It provides the services and features present in
abstract views of all its users through the computer system.It also enables the services and features to evolve
over time as users' needs change.People who design operating systems have to deal with three issues: efficient use
of the computer system's resources, the convenience of users, and prevention of interference with users'
activities. Efficient use is more important when a computer system is dedicated to specific applications, and user
convenience is more important in personal computers, while both are equally
importantwhenacomputersystemissharedbyseveralusers.Hence,the designer aims for the right combination of efficient
use and user convenience for the operating system's environment. Prevention of interference is mandatory in all
environments.We will now take a broad look at what makes an operating system work—we will see how its functions of
program management and resource management help to ensure efficient use of resources and user convenience, and how
the functions of security and protection prevent interference with programs and resources";
                document.getElementById('previewSection').style.display = 'block';
            }
        }

        document.querySelectorAll('.option-card').forEach(card => {
            card.addEventListener('click', () => {
                const selector = card.dataset.type ? '.option-card[data-type]' : '.option-card[data-mode]';
                document.querySelectorAll(selector).forEach(c => c.classList.remove('active'));
                card.classList.add('active');
            });
        });

        document.getElementById('fileInput').addEventListener('change', (e) => {
            const file = e.target.files[0];
```

```
        if (file) {
            const reader = new FileReader();
            reader.onload = event => document.getElementById('textInput').value = event.target.result;
            reader.readAsText(file);
        }
    });

    async function generateSummary() {
        const text = document.getElementById('textInput').value.trim();
        const fileInput = document.getElementById('fileInput');
        const file = fileInput.files[0];

        if (!text && !file) {
            alert('Please enter text or upload a file!');
            return;
        }

        document.getElementById("popupModal").style.display = "block";

        const summaryType = document.querySelector('.option-card[data-type].active').dataset.type;

        const btn = document.getElementById('generateBtn');
        btn.disabled = true;
        btn.innerHTML = '<span class="loading"></span> Generating...';

        try {
            const formData = new FormData();
            formData.append('summary_type', summaryType);

            if (text) {
                formData.append('text', text);
            }

            if (file) {
                formData.append('file', file);
            }

            const response = await fetch('http://localhost:8000/api/summarize', {
                method: 'POST',
                body: formData
            });

            const result = await response.json();

            if (result.success) {
                document.getElementById('results').innerHTML = `
            <div class="results-card">
                <h3>Extractive (${result.summary_type}) Summary</h3>
                <div class="results-text">${result.summary}</div>
                <div style="margin-top:15px;font-size:14px;color:#666;">
                    ${result.original_length} words → ${result.summary_length} words
                </div>
            </div>`;
                document.getElementById('results').style.display = 'block';
            } else {
                alert(result.error || "Error generating summary");
            }

        } catch (error) {
            alert('Error: ' + error.message);
        } finally {
            document.getElementById("popupModal").style.display = "none";
```

```
                btn.disabled = false;
                btn.innerHTML = 'Generate Summary';
            }
        }
        function openAbout() {
            document.getElementById("aboutModal").style.display = "block";
        }

        function closeAbout() {
            document.getElementById("aboutModal").style.display = "none";
        }
        function openTechDoc() {
            document.getElementById("techDocModal").style.display = "block";
        }

        function closeTechDoc() {
            document.getElementById("techDocModal").style.display = "none";
        }
        function showDemo() { document.getElementById("demoModal").style.display = "block"; }
        function closeDemo() { document.getElementById("demoModal").style.display = "none"; }

        document.addEventListener('keypress', (e) => {
            if (e.key === 'Enter' && document.getElementById('loginModal').style.display !== 'none') login();
        });

    </script>
</body>

</html>
```

app.py(Backend)

```
from fastapi import FastAPI, File, UploadFile, Form, HTTPException, Request
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import HTMLResponse, JSONResponse
from fastapi.staticfiles import StaticFiles
from starlette.middleware.base import BaseHTTPMiddleware
import uvicorn
import re
import os
import tempfile
from pathlib import Path
import warnings
from collections import Counter
import string
import math

warnings.filterwarnings("ignore")

# ================ CONFIG ================

MAX_FILE_SIZE = 5 * 1024 * 1024  # 5MB
ALLOWED_EXTENSIONS = {".txt", ".docx"}
MAX_INPUT_WORDS = 1000

# Basic stopwords list (can expand if needed)
STOPWORDS = {
    "the", "is", "in", "and", "to", "of", "a", "an", "on", "for",
```

```
    "with", "as", "by", "at", "from", "that", "this", "it", "or",
    "are", "be", "was", "were", "has", "have", "had", "but", "not",
    "which", "their", "its", "can", "will", "would", "should"
}

# ================ APP SETUP ================

app = FastAPI(title="AI Summarizer - Secure Production (Extractive Only)")
app.mount("/static", StaticFiles(directory="static"), name="static")


# ================ SECURITY HEADERS ================

class SecurityHeadersMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next):
        response = await call_next(request)
        response.headers["X-Content-Type-Options"] = "nosniff"
        response.headers["X-Frame-Options"] = "DENY"
        response.headers["X-XSS-Protection"] = "1; mode=block"
        response.headers["Strict-Transport-Security"] = "max-age=31536000"
        return response


app.add_middleware(SecurityHeadersMiddleware)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["POST", "GET"],
    allow_headers=["*"],
)

# ================ TEXT CLEANING ================

def clean_text(text: str) -> str:
    if not text:
        return ""

    text = re.sub(r"\s+", " ", text.strip())

    words = text.split()
    if len(words) > MAX_INPUT_WORDS:
        words = words[:MAX_INPUT_WORDS]

    return " ".join(words)


# ================ ADVANCED EXTRACTIVE SUMMARIZER ================

def tokenize_sentences(text: str):
    return [s.strip() for s in re.split(r'(?<=[.!?])\s+', text) if s.strip()]


def tokenize_words(text: str):
    text = text.lower().translate(str.maketrans("", "", string.punctuation))
    return [w for w in text.split() if w not in STOPWORDS]


def compute_tf(sentences):
    tf_scores = []
    for sentence in sentences:
```

```
        words = tokenize_words(sentence)
        word_freq = Counter(words)
        total_words = len(words) if words else 1
        tf_scores.append({word: freq / total_words for word, freq in word_freq.items()})
    return tf_scores


def compute_idf(sentences):
    N = len(sentences)
    idf = {}
    all_words = set(word for sentence in sentences for word in tokenize_words(sentence))

    for word in all_words:
        containing = sum(1 for sentence in sentences if word in tokenize_words(sentence))
        idf[word] = math.log((N + 1) / (containing + 1)) + 1

    return idf


# ================ IMPROVED KEY-POINT EXTRACTIVE SUMMARIZER ================

def extractive_summary(text: str, num_sentences: int = 3) -> str:
    sentences = tokenize_sentences(text)

    if len(sentences) <= num_sentences:
        return " ".join(sentences)

    # Compute global word frequencies
    words = tokenize_words(text)
    word_freq = Counter(words)

    if not word_freq:
        return " ".join(sentences[:num_sentences])

    max_freq = max(word_freq.values())

    # Normalize frequencies
    for word in word_freq:
        word_freq[word] /= max_freq

    sentence_scores = {}

    for sentence in sentences:
        sentence_words = tokenize_words(sentence)

        if len(sentence_words) < 4:
            continue  # ignore very short sentences

        score = 0
        for word in sentence_words:
            score += word_freq.get(word, 0)

        # Normalize by sentence length to prevent long sentence bias
        score = score / len(sentence_words)

        sentence_scores[sentence] = score

    # Select top scoring sentences
    ranked_sentences = sorted(sentence_scores.items(),
                              key=lambda x: x[1],
                              reverse=True)
```

```
    selected = ranked_sentences[:num_sentences]

    # Preserve original order
    selected_sentences = sorted(
        [s[0] for s in selected],
        key=lambda s: sentences.index(s)
    )

    return " ".join(selected_sentences)

# ================ FILE HANDLING ================

def extract_text_from_file(file_path: str) -> str:
    try:
        ext = Path(file_path).suffix.lower()

        if ext not in ALLOWED_EXTENSIONS:
            return ""

        if ext == ".txt":
            with open(file_path, "r", encoding="utf-8", errors="ignore") as f:
                return f.read()

        elif ext == ".docx":
            import docx
            doc = docx.Document(file_path)
            return "\n".join(p.text for p in doc.paragraphs if p.text.strip())

        return ""
    except:
        return ""


# ================ API ENDPOINT ================

@app.post("/api/summarize")
async def summarize_api(
    text: str = Form(""),
    file: UploadFile = File(None),
    summary_type: str = Form("short"),
):
    try:
        input_text = text

        if file and file.filename:
            suffix = Path(file.filename).suffix.lower()

            if suffix not in ALLOWED_EXTENSIONS:
                raise HTTPException(status_code=400, detail="Unsupported file type")

            contents = await file.read()

            if len(contents) > MAX_FILE_SIZE:
                raise HTTPException(status_code=400, detail="File too large")

            with tempfile.NamedTemporaryFile(delete=False, suffix=suffix) as tmp:
                tmp.write(contents)
                tmp_path = tmp.name

            input_text = extract_text_from_file(tmp_path)
            os.unlink(tmp_path)
```

```
        if not input_text.strip():
            raise HTTPException(status_code=400, detail="No text provided")

        clean_input = clean_text(input_text)

        total_sentences = len(tokenize_sentences(clean_input))
        if summary_type == "short":
            num_sentences = max(2, int(total_sentences * 0.15))
        elif summary_type == "medium":
            num_sentences = max(3, int(total_sentences * 0.30))
        elif summary_type == "large":
            num_sentences = max(5, int(total_sentences * 0.50))
        else:
            num_sentences = max(2, int(total_sentences * 0.15))
# Ensure we never exceed total sentences
        num_sentences = min(num_sentences, total_sentences)
        summary = extractive_summary(clean_input, num_sentences)
        summary = summary.strip()
        if summary and not summary.endswith(('.', '!', '?')):
            summary += '.'
        return {
            "success": True,
            "summary": summary,
            "summary_type": summary_type,
            "mode": "Extractive",
            "original_length": len(clean_input.split()),
            "summary_length": len(summary.split())
        }

    except HTTPException:
        raise

    except Exception as e:
        raise HTTPException(
            status_code=500,
            detail=str(e)
        )


# ================ FRONTEND ROUTE =================

@app.get("/", response_class=HTMLResponse)
async def frontend():
    try:
        with open("index.html", "r", encoding="utf-8") as f:
            return f.read()
    except:
        return HTMLResponse("""
        <h1>AI Extractive Summarizer</h1>
        <p>Server Working!</p>
        <p><a href="/health">Health check</a> | <a href='/docs'>API Docs</a></p>
        """)


@app.get("/health")
async def health():
    return {"status": "healthy", "mode": "extractive-only", "error": "none"}


# ================ RUN SERVER =================

if __name__ == "__main__":
```

```
    uvicorn.run(
        app,
        host="0.0.0.0",
        port=8000,
        log_level="info"
    )
```